Portfolio Developer Dashboard

Technical Documentation

Version 1.0 — March 2026

Built with React · JavaScript · Vite · Tailwind CSS

Table of Contents

1. Project Overview

2. Technology Stack

3. Application Architecture

4. Routes & Navigation

5. Components

6. API Services

7. Context & State Management

8. Features Summary

9. Usage Examples

10. Dependencies

1. Project Overview

The Portfolio Developer Dashboard is a single-page application (SPA) designed for developers to manage projects, track tasks via a Kanban-style board, log timesheet entries, and explore GitHub profiles with repository and commit analytics.

The application features a responsive design with full dark mode support, drag-and-drop task management, interactive data visualizations, and integration with the GitHub REST API.

2. Technology Stack

Technology	Version	Purpose
React	^18.3.1	UI framework
Vite	^8.0.0	Build tool & dev server
Tailwind CSS	^3.4.17	Utility-first CSS framework
React Router DOM	^6.30.1	Client-side routing
Recharts	^2.15.4	Charting & data visualization
Pragmatic DnD	^1.7.9	Drag-and-drop for task board
date-fns	^3.6.0	Date utility functions
Lucide React	^0.462.0	Icon library

3. Application Architecture

The application follows a component-based architecture with the following structure:

```
src/
  App.tsx                    # Root component with providers
  main.tsx                   # Entry point
  index.css                  # Global styles & design tokens
  components/
    Navbar.tsx               # Navigation + routing layout
```

```
    ProjectList.tsx         # CRUD project management
    TaskBoard.tsx           # Kanban drag-and-drop board
    ActivityChart.tsx       # Timesheet & analytics
    GithubProfile.tsx       # GitHub API integration
  context/
    ThemeContext.tsx         # Dark/light theme provider
  services/
    GithubAPI.ts            # GitHub REST API functions
  pages/
    Dashboard.tsx           # Home page (renders ProjectList)
    Projects.tsx            # Projects page (renders GithubProfile)
```

Provider Hierarchy

```
ThemeProvider > BrowserRouter > Navbar
```

4. Routes & Navigation

All routes are defined inside the Navbar component which provides a persistent navigation bar with responsive mobile hamburger menu.

Route Path	Component	Description
/	Dashboard → GithubProfile	GitHub profile explorer
/projects	Projects → ProjectList	Home page with project management
/tasks	TaskBoard	Kanban-style task board
/activity	ActivityChart	Timesheet & analytics dashboard

Navigation Features

- Responsive navbar with desktop horizontal links and mobile hamburger menu
- Active route highlighting with blue background indicator

- Theme toggle button (dark/light mode) in both desktop and mobile views
- Sticky top navigation with z-50 stacking

5. Components

5.1 ProjectList

A full CRUD project manager rendered on the Projects page.

State

- projects: Array of project objects (id, name, description, techStack, status)
- newProject: Form state for adding/editing
- editingId: Tracks which project is being edited

Functions

Function	Parameters	Description
handleAdd()	—	Adds a new project to the list
handleEdit(id)	id: number	Loads project data into form for editing
handleUpdate()	—	Saves edited project changes
handleDelete(id)	id: number	Removes a project from the list

Features

- Card-based grid layout (1/2/3 columns responsive)
- Read More/Read Less toggle for descriptions exceeding 20 characters
- Status badges (In Progress / Completed) with color coding
- Tech stack tags rendered as comma-separated chips
- Empty state with dashed border placeholder

5.2 TaskBoard

A Kanban-style task board with three columns: TO DO, IN PROGRESS, and DONE. Supports full drag-and-drop using Atlassian's Pragmatic Drag and Drop library.

State

- tasks: Object with keys { todo, inProgress, done }, each containing task arrays
- dialogOpen / editingTask / targetColumn / formData: Modal form management
- expandedDescs: Tracks read-more state per task

Functions

Function	Parameters	Description
handleDrop(column, taskId)	column, taskId	Moves task between columns on drag-drop
openAddDialog(column)	column: string	Opens modal to add task to a column
openEditDialog(task, column)	task, column	Opens modal pre-filled for editing
handleSubmit()	—	Creates or updates task from modal form
deleteTask(taskId, column)	taskId, column	Removes task from specified column
toggleDesc(id)	id: number	Toggles read-more on task description

Features

- Drag-and-drop between columns using @atlaskit/pragmatic-drag-and-drop
- Modal dialog for Add/Edit with title, description, and assignee fields
- Assignee avatar with initial letter badge
- Hover-reveal edit/delete action buttons
- Column task count badges with color coding
- Read more/less for long descriptions

5.3 ActivityChart (Timesheet)

A consulting-style timesheet dashboard for logging daily work activity with full CRUD support and multiple chart visualizations.

State

- entries: Array of timesheet entries (date, hours, tasks, projects, category, notes)
- viewMode: ‘weekly’ or ‘monthly’ toggle
- currentDate: Navigation anchor for date range
- formData / showForm / editingId: Entry form management

Functions

Function	Parameters	Description
navigate(dir)	dir: 1 | -1	Moves date range forward/backward
handleAdd()	—	Saves a new timesheet entry
handleDelete(id)	id: number	Removes a timesheet entry
handleEditSave(id, field, value)	id, field, value	Inline-edits a field on an entry

Charts

- Bar Chart: Hours, tasks & projects breakdown per day/week
- Pie Chart: Hours distribution by category (Development, Meetings, etc.)
- Area Chart: Cumulative actual hours vs. target line
- Line Chart: Daily hours trend

KPI Cards

- Total Hours, Tasks Done, Projects, Avg Hours/Day, Utilization %

Smart Insights

- Utilization warnings (high/low)
- Peak day identification

- Top category analysis
- Burnout risk detection (avg > 8h/day)

5.4 GithubProfile

Integrates with the GitHub REST API to fetch and display user profiles, repositories, and commit counts.

State

- username: Search input for GitHub username
- userData: Fetched GitHub user profile object
- repos: Array of repository objects for current page
- commits: Array of { repo, commits } commit count data
- page / totalPages: Pagination state

Features

- User profile card with followers, public repos, total stars, total commits
- Repository cards grid with commit counts
- Paginated repository browsing (100 repos per page)

6. API Services

All GitHub API interactions are centralized in src/services/GithubAPI.ts.

6.1 fetchUserData

Endpoint: GET https://api.github.com/users/{username}

Parameters: username (string)

Returns: GitHub user profile object (name, login, followers, public_repos, etc.)

Throws: Error if user not found (non-200 response)

Example

```
const user = await fetchUserData('octocat');
console.log(user.name);      // 'The Octocat'
console.log(user.followers);  // 9441
```

6.2 fetchUserRepos

Endpoint: GET https://api.github.com/users/{username}/repos?per_page=100&page={page}

Parameters: username (string), page (number)

Returns: Array of repository objects

Throws: Error on fetch failure

Example

```
const repos = await fetchUserRepos('octocat', 1);
console.log(repos.length);           // up to 100
console.log(repos[0].stargazers_count); // star count
```

6.3 fetchUserCommits

Endpoint: GET https://api.github.com/repos/{username}/{repo}/commits?per_page=1

Parameters: username (string), repos (array of repo objects)

Returns: Array of { repo: string, commits: number }

Logic: Uses the Link header pagination to determine total commit count without fetching all commits.

Example

```
const commits = await fetchUserCommits('octocat', repos);
// [{ repo: 'Hello-World', commits: 42 }, ...]
```

7. Context & State Management

7.1 ThemeContext

File: src/context/ThemeContext.tsx

Provides application-wide dark/light theme toggling with localStorage persistence.

API

Export	Type	Description
ThemeProvider	Component	Wraps app; manages theme state and applies dark class to <html>
useTheme()	Hook	Returns { theme: 'light' | 'dark', toggleTheme: () => void }

Behavior

- Reads initial theme from localStorage (defaults to ‘light’)
- Toggles ‘dark’ class on document.documentElement
- Persists theme preference to localStorage on every change

8. Features Summary

Feature	Component	Details
Project Management	ProjectList	Full CRUD with card-based UI, status badges, tech stack tags
Task Board	TaskBoard	Kanban with drag-and-drop, modal forms, assignee tracking
Timesheet & Analytics	ActivityChart	Time logging, 4 chart types, KPI cards, smart insights
GitHub Integration	GithubProfile	Profile search, repo/commit analytics, pagination

Dark Mode	ThemeContext	System-wide toggle with localStorage persistence
Responsive Design	Navbar + All	Mobile hamburger menu, responsive grids, overflow scroll
Drag & Drop	TaskBoard	Atlassian Pragmatic DnD for cross-column task movement

9. Usage Examples

Adding a Project

1. Navigate to the Projects page (/projects)
2. Fill in the project name, description, tech stack, and select status
3. Click "Add" to create the project card
4. Hover over a card to reveal Edit and Delete actions

Managing Tasks

5. Navigate to /tasks
6. Click "+ Add Task" in any column (TO DO, IN PROGRESS, DONE)
7. Fill in title, description, and assignee in the modal
8. Drag tasks between columns to update their status

Logging Timesheet Hours

9. Navigate to /activity
10. Click "Log Time" to open the entry form
11. Enter date, hours, tasks completed, projects, category, and notes
12. Switch between Weekly and Monthly views using the toggle
13. Navigate date ranges using the arrow buttons

Exploring GitHub Profiles

14. Navigate to /
15. Enter a GitHub username in the search box

16. Click "Fetch Profile" to load user data
17. Browse repository cards with commit counts; use pagination for large accounts

10. Dependencies

Runtime Dependencies

Package	Purpose
react, react-dom	Core UI framework
react-router-dom	Client-side SPA routing
recharts	Data visualization (Bar, Pie, Area, Line charts)
@atlaskit/pragmatic-drag-and-drop	Native drag-and-drop for task board
date-fns	Date manipulation & formatting
lucide-react	SVG icon library
tailwindcss	CSS utility helpers

Dev Dependencies

Package	Purpose
vite + @vitejs/plugin-react	Build tooling & HMR
tailwindcss	CSS processing pipeline
vitest + @testing-library/react	Unit testing framework
@playwright/test	End-to-end testing
eslint + plugins	Code quality enforcement

— End of Documentation —